
Manual de Usuario

Proyecto 1: Autómatas Celulares

Life-Like • Hormiga de Langton

Tecnológico de Costa Rica

Escuela de Ingeniería en Computación

IC-1801 Taller de Programación

Prof. Mauricio Avilés

6 de junio de 2026

Este manual explica **paso a paso** cómo instalar, configurar y utilizar los dos programas de autómatas celulares desarrollados en Python: el **autómata Life-Like** (generalización del Juego de la Vida de Conway) y la **Hormiga de Langton** generalizada.

Índice

1. Descripción del proyecto	3
1.1. Aplicaciones de los autómatas celulares	3
2. Requisitos del sistema	3
3. Instalación de Python	3
3.1. En Windows	4

3.2. En macOS	4
3.3. En Linux (Ubuntu/Debian)	5
4. Instalación de las bibliotecas necesarias	5
5. Descarga del programa desde GitHub	5
5.1. Opción A: Descargar el ZIP completo (recomendado)	6
5.2. Opción B: Descargar archivos individuales	6
6. Cómo ejecutar el programa	6
6.1. En Windows	7
6.2. En macOS	7
6.3. En Linux	8
7. Uso del autómata Life-Like	8
7.1. Pantalla de configuración inicial	8
7.2. Controles durante la simulación	9
7.3. Interacción con el mouse	9
7.4. Guardar y cargar estados	9
7.5. Ejemplos de reglas Life-Like	10
8. Uso de la Hormiga de Langton	10
8.1. Pantalla de configuración inicial	10
8.2. Cómo funcionan las reglas	10
8.3. Controles durante la simulación	11
8.4. Comportamientos emergentes	11
8.5. Ejemplos de strings de instrucciones	11
9. Solución de problemas comunes	12
10. Resumen de controles	12
11. Créditos y referencias	13

1 Descripción del proyecto

Los autómatas celulares son modelos computacionales que consisten en una cuadrícula de células. Cada célula tiene un estado y evoluciona en el tiempo según reglas que consideran el estado de sus vecinos. Este proyecto implementa dos tipos:

- **Life-Like:** Generalización del Juego de la Vida de Conway. Las células tienen dos estados (viva/muerta) y evolucionan según reglas de nacimiento y supervivencia configurables por el usuario en notación Bx/Sy .
- **Hormiga de Langton:** Una abstracción llamada “hormiga” recorre una cuadrícula girando a la izquierda o derecha según el color de la celda donde se encuentra. Soporta múltiples colores definidos por un string de L's y R's.

1.1 Aplicaciones de los autómatas celulares

El estudio de autómatas celulares tiene aplicaciones en simulaciones de tráfico, diseño de estructuras, composición musical, criptografía, generación de números aleatorios, vida artificial, y modelado de fenómenos naturales como el crecimiento de cristales y la evolución.

2 Requisitos del sistema

- **Sistema operativo:** Windows 10/11, macOS 10.15+, o Linux (Ubuntu/Debian recomendado)
- **Python:** versión 3.8 o superior
- **Bibliotecas Python:** pygame, easygui
- **Conexión a internet:** necesaria para descargar el programa desde GitHub

3 Instalación de Python

Si ya tienes Python instalado, puedes verificarlo abriendo una terminal y escribiendo `python --version` o `python3 --version`. Si aparece un número de versión como `Python 3.x.x`, puedes saltar directamente a la Sección 5.

3.1 En Windows

1. Abre Google Chrome, Edge o cualquier navegador web.
2. Ve a <https://www.python.org/downloads/>
3. Haz clic en el botón “**Download Python**” (en amarillo, descargará la versión más reciente).
4. Abre el archivo descargado (se llama algo como `python-3.x.x-amd64.exe`).
5. ★ **MUY IMPORTANTE:** Antes de hacer clic en *Install Now*, asegúrate de marcar la casilla que dice “**Add Python to PATH**” en la parte inferior de la ventana. Sin esto el programa no funcionará.
6. Haz clic en “**Install Now**” y espera unos 2 minutos.
7. Cuando aparezca “Setup was successful”, cierra el instalador.

Verificar la instalación en Windows

1. Presiona **Tecla Windows + R**, escribe `cmd` y presiona Enter.
2. En la ventana negra que aparece, escribe:

```
python --version
```

Deberías ver algo como `Python 3.12.0`. Si aparece ese mensaje, Python está correctamente instalado.

3.2 En macOS

1. Abre **Terminal**: presiona **Command + Espacio**, escribe “Terminal” y presiona Enter.
2. Primero instala Homebrew (gestor de paquetes para Mac). Copia y pega este comando completo:

```
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

3. Espera a que termine (puede tardar varios minutos, te pedirá tu contraseña).
4. Luego instala Python con:

```
brew install python
```

3.3 En Linux (Ubuntu/Debian)

1. Abre una terminal con **Ctrl + Alt + T**.
2. Escribe los siguientes comandos (uno por uno, presionando Enter después de cada uno):

```
sudo apt update  
sudo apt install python3 python3-pip
```

4 Instalación de las bibliotecas necesarias

El programa requiere **Pygame** (para la visualización gráfica) y **easygui** (para los cuadros de diálogo). Una vez que Python esté instalado, abre una terminal o símbolo del sistema y ejecuta:

En Windows (CMD)

```
pip install pygame easygui
```

En macOS / Linux

```
pip3 install pygame easygui
```

Si aparece “pip no se reconoce” en Windows: Esto indica que Python no fue agregado al PATH. Reinstala Python marcando la opción **Add Python to PATH**.

Si aparece error de permisos en Linux: Agrega **--user** al comando: `pip3 install --user pygame easygui`

5 Descarga del programa desde GitHub

No necesitas instalar Git para descargar el proyecto. Puedes descargar directamente desde el navegador siguiendo estos pasos.

5.1 Opción A: Descargar el ZIP completo (recomendado)

1. Abre tu navegador y ve a la página principal del repositorio en GitHub.
2. Haz clic en el botón verde que dice “<> Code”.
3. En el menú que aparece, selecciona “Download ZIP”.
4. Guarda el archivo ZIP en tu **Escritorio** o en una carpeta fácil de encontrar.
5. Haz clic derecho sobre el archivo ZIP y selecciona “**Extraer aquí**” (Windows) o “**Descomprimir**” (Mac/Linux).
6. Aparecerá una carpeta con todos los archivos del proyecto.

5.2 Opción B: Descargar archivos individuales

1. Abre el enlace del archivo .py en GitHub.
2. Busca el botón “**Raw**” en la parte superior derecha del código.
3. Haz clic derecho sobre “**Raw**” y selecciona “**Guardar enlace como...**”.
4. Guarda el archivo en tu **Escritorio** con extensión .py.
5. **Importante:** Asegúrate de que el archivo termine en .py y no en .py.txt. Si Windows agregó .txt al final, renombra el archivo eliminando esa parte.

El archivo se guardó como .txt: En Windows, abre el Explorador de archivos, haz clic en la pestaña “Ver” y activa “Extensiones de nombre de archivo”. Luego renombra el archivo eliminando la extensión .txt.

6 Cómo ejecutar el programa

6.1 En Windows

1. Presiona **Tecla Windows**, escribe CMD y abre **Símbolo del sistema**.
2. Navega a la carpeta donde guardaste el programa. Por ejemplo, si lo guardaste en el Escritorio:

```
cd %USERPROFILE%\Desktop
```

3. Ejecuta el autómata Life-Like:

```
python life_like.py
```

4. O ejecuta la Hormiga de Langton:

```
python hormiga_langton.py
```

6.2 En macOS

1. Abre **Terminal** (Command + Espacio, escribe “Terminal”).
2. Navega a la carpeta del proyecto:

```
cd ~/Desktop
```

3. Ejecuta el programa deseado:

```
python3 life_like.py  
python3 hormiga_langton.py
```

6.3 En Linux

1. Abre Terminal con **Ctrl + Alt + T**.

2. Navega al directorio del proyecto:

```
cd ~/Escritorio
```

3. Ejecuta el programa:

```
python3 life_like.py  
python3 hormiga_langton.py
```

7 Uso del autómata Life-Like

El autómata Life-Like es una generalización del Juego de la Vida de Conway. Las reglas se expresan en notación **Bx/Sy** (B = birth/nacimiento, S = survival/supervivencia).

7.1 Pantalla de configuración inicial

Al ejecutar el programa, aparecerá una **ventana de configuración** (cuadro de diálogo gráfico con easygui) que solicita los siguientes parámetros:

- **Número de filas:** cantidad de filas de la cuadrícula (p.ej. 50).
- **Número de columnas:** cantidad de columnas (p.ej. 80).
- **Tamaño de celda:** tamaño en píxeles de cada célula (p.ej. 10).
- **Reglas de nacimiento (B):** dígitos del 0 al 8 que indican con cuántos vecinos vivos nace una célula muerta. Ejemplo: 3 para Conway.
- **Reglas de supervivencia (S):** dígitos del 0 al 8 que indican con cuántos vecinos vivos sobrevive una célula viva. Ejemplo: 23 para Conway.

Para reproducir el **Juego de la Vida de Conway** clásico, introduce:

- Nacimiento (B): 3
- Supervivencia (S): 23

Esto representa la regla **B3/S23**.

7.2 Controles durante la simulación

Una vez iniciada la simulación, la ventana de Pygame muestra la cuadrícula en tiempo real. Los controles disponibles son:

Tecla / Acción	Función
Espacio	Pausar o reanudar la simulación
G	Guardar el estado completo en un archivo (usando pickle)
C	Cargar un estado previamente guardado desde archivo
R	Reiniciar la cuadrícula con valores aleatorios
B	Reiniciar la cuadrícula con valores neutros (todo en ceros)
Clic izquierdo	Cambiar el estado de una célula al siguiente estado posible

7.3 Interacción con el mouse

Mientras la simulación está **pausada** o en ejecución, puedes hacer clic sobre cualquier célula de la cuadrícula para cambiar su estado. En el autómata Life-Like, los dos estados son:

- **Muerta (0):** célula inactiva, se muestra en negro.
- **Viva (1):** célula activa, se muestra en el color configurado.

Cada clic alterna la célula al siguiente estado en el ciclo.

7.4 Guardar y cargar estados

Guardar (tecla G)

Al presionar **G**, el programa guarda automáticamente en un archivo (usando el módulo `pickle` de Python) el estado completo de la cuadrícula, incluyendo:

- El estado de todas las células.
- Las dimensiones (filas y columnas).
- El tamaño de cada célula.
- Los parámetros del autómata (reglas B y S).

Cargar (tecla C)

Al presionar **C**, aparece un cuadro de diálogo para seleccionar el archivo previamente guardado. El programa restaurará exactamente el estado que fue guardado.

7.5 Ejemplos de reglas Life-Like

A continuación se presentan algunas reglas predefinidas con comportamientos interesantes que puedes probar:

Nombre	Regla	Comportamiento
Conway's Life	B3/S23	Patrones estables y osciladores clásicos
Replicator	B1357/S1357	El patrón inicial se replica formando copias
Life without Death	B3/S012345678	Células nunca mueren, crecen en escalera
Diamoeba	B35678/S5678	Forma diamantes grandes con bordes fluctuantes
34 Life	B34/S34	Patrones similares a Conway que explotan
Anneal	B4678/S35678	Curvas que convergen hacia la más corta

8 Uso de la Hormiga de Langton

La Hormiga de Langton es un autómata en el que una “hormiga” virtual recorre una cuadrícula. El comportamiento emerge de dos reglas simples que dependen del color de la celda donde se ubica la hormiga.

8.1 Pantalla de configuración inicial

Al ejecutar el programa, la ventana de configuración solicita:

- **Número de filas:** cantidad de filas de la cuadrícula.
- **Número de columnas:** cantidad de columnas.
- **Tamaño de celda:** tamaño en píxeles de cada célula.
- **String de instrucciones:** una cadena de letras L (izquierda) y R (derecha) que indica la acción de la hormiga para cada color. La longitud del string determina la cantidad de colores. Ejemplo: LR para la hormiga clásica de dos colores.

Para la **Hormiga de Langton clásica** (dos colores: blanco y negro):

- String de instrucciones: LR

Esto significa: gira a la izquierda en celdas del primer color, gira a la derecha en celdas del segundo color.

8.2 Cómo funcionan las reglas

Las reglas de la hormiga son:

1. Si la hormiga está en una celda cuyo color corresponde a una **L** en el string: gira 90° a la **izquierda**, cambia el color de la celda al siguiente en el ciclo, y avanza una celda.
2. Si corresponde a una **R**: gira 90° a la **derecha**, cambia el color al siguiente, y avanza.

8.3 Controles durante la simulación

Los controles son idénticos a los del autómata Life-Like:

Tecla / Acción	Función
Espacio	Pausar o reanudar la simulación
G	Guardar estado completo en archivo
C	Cargar estado desde archivo
R	Reiniciar con valores aleatorios
B	Reiniciar con valores neutros (ceros)
Clic izquierdo	Cambiar el estado/color de una célula

8.4 Comportamientos emergentes

Si se inicia la cuadrícula toda de un color y se ubica la hormiga en el centro, se observarán tres fases:

1. **Simplicidad:** durante los primeros cientos de pasos, la hormiga crea patrones simples y a menudo simétricos.
2. **Caos:** después de cierto punto emerge un patrón irregular que continúa hasta aproximadamente el paso 10000.
3. **Orden emergente:** finalmente la hormiga construye una “autopista”, un patrón de 104 pasos que se repite indefinidamente.

8.5 Ejemplos de strings de instrucciones

String	Colores	Comportamiento
LR	2	Hormiga clásica: caos → autopista
RLR	3	Crecimiento caótico
LLRR	4	Patrones simétricos
LRRRRLLR	9	Crece en forma de cuadrado
RLLLLRLLRR	10	Caótico inicial, luego triángulo creciente
LLRLLLLRRRRR	12	Caótico en el centro, triángulos alrededor

9 Solución de problemas comunes

“python no se reconoce como comando” (Windows)

Reinstala Python marcando la opción **Add Python to PATH** durante la instalación.

“No module named pygame” o “No module named easygui”

Instala las bibliotecas con: `pip install pygame easygui` (Windows) o `pip3 install pygame easygui` (Mac/Linux).

El archivo se guardó como `.py.txt`

En el Explorador de Windows, activa la vista de extensiones de archivo y renombra eliminando el `.txt` final.

La ventana del programa se cierra inmediatamente

Ejecuta el programa desde la terminal (CMD o Terminal) para ver el mensaje de error. Normalmente indica un paquete faltante.

No encuentro el botón “Raw” en GitHub

Usa el enlace directo al archivo raw: agrega `raw.` antes de `githubusercontent.com` en la URL, o busca el botón en la parte superior derecha del visor de código de GitHub.

10 Resumen de controles

Tecla	Acción	Descripción
Espacio	Pausar / Reanudar	Detiene o continúa la simulación
G	Guardar	Guarda estado en archivo pickle
C	Cargar	Carga estado desde archivo pickle
R	Reset aleatorio	Reinicia con células al azar
B	Reset neutro	Reinicia con todo en cero
Clic	Editar célula	Cambia el estado de la célula clicada

11 Créditos y referencias

- Cellular automaton. (2016, May 19). In *Wikipedia, The Free Encyclopedia*. https://en.wikipedia.org/w/index.php?title=Cellular_automaton
- Langton's ant. (2015, December 16). In *Wikipedia, The Free Encyclopedia*. https://en.wikipedia.org/w/index.php?title=Langton%27s_ant
- LifeWiki. (2025, June 9). https://conwaylife.com/wiki/Main_Page
- Proyecto desarrollado para **IC-1801 Taller de Programación**, Tecnológico de Costa Rica.
Profesor: Mauricio Avilés.

¡Listo! Ya puedes explorar los autómatas celulares.

Recuerda: cada simulación puede mostrar comportamientos únicos e inesperados.